Jones Automation — Exercise

Levi Silva Freitas

Billing UI Mock-up Analysis

Scope of Submission

This document covers the second part of the exercise, focusing on the analysis of the billing UI mock-up. The submission addresses the following questions:

A. **UI Review:** Identify potential problems with the screen, considering functionality, security, usability, and performance.

B. **Sample Test Cases:** Provide 2–3 sample test cases for the billing functionality.

C. **Product Solution:** Propose a product solution for the most severe issue identified.

A. UI Review — Identified Problems

Observable Issues

1. Payment Amount Is Displayed Without a Currency Indicator
Category: Usability / Functional Clarity — Observable Issue

The payment amount is displayed as 30.00, but no currency code or symbol is provided. Customers may interpret the amount differently depending on their location or expectations.

Recommendation: Display the amount with an unambiguous currency identifier, such as USD 30.00, and ensure it matches the currency used for the actual transaction.

2. Billing Address Fields Lack Clear Individual Labels
Category: Usability / Accessibility — Observable Issue

Under Credit Card Billing Street Address, two input fields are displayed, but only one shared label is visible. Users may not understand the intended purpose of the second field.

Recommendation: Provide explicit labels or examples, such as Street Address and Apartment, Suite, or Unit (optional), if these are the intended purposes.

3. The Meaning of the MI Field Is Not Explained
Category: Usability — Observable Issue

The cardholder name section uses the abbreviation MI without explaining its meaning. Some users may not recognize that it refers to a middle initial.

Recommendation: Replace the abbreviation with a clear label, such as Middle Initial, or provide a descriptive hint.

Potential Functional and Security Risks

4. Risk of Duplicate Payment Submissions
Category: Functional Reliability / Financial Risk — Potential Issue

The screen provides a Continue button, but the mock-up does not specify how repeated clicks are handled. Users may click multiple times if the interface does not respond immediately, potentially resulting in duplicate payment requests.

Recommendation: Prevent repeated submissions through frontend interaction controls and backend safeguards, such as idempotency mechanisms.

5. Sensitive Payment Information Requires Appropriate Protection
Category: Security — Integration Risk

The form collects sensitive information, including the card number and cardholder details. The mock-up does not provide evidence of how this information is transmitted, processed, or stored.

Recommendation: Ensure that payment information is handled securely, avoid exposing sensitive data in logs, and use a payment-processing flow that meets applicable security requirements, including PCI DSS where relevant.

Requirements to Define Before Integration

6. Field Validation Requirements Are Not Specified
Category: Functional Correctness / Usability — Requirement to Define

The form contains fields for card information, expiration date, cardholder name, billing address, and postal code, but the mock-up does not specify the validation rules or how invalid inputs should be handled.

Recommendation: Define required fields, accepted formats, card-number validation, expiration-date rules, and error messages. Specify which fields are mandatory and validate data on both the client and server sides.

7. Supported Countries and Address Validation Rules Are Unclear
Category: Functional Correctness / Internationalization — Requirement to Define

The form includes a State or Province field and a Postal Code field, suggesting that it may support multiple regions. However, the mock-up does not specify which countries are supported or how address fields should behave for different locations.

Recommendation: Define the supported countries and their address formats. Adapt state or province options and postal-code validation to the selected country, where applicable.

8. Payment Submission and Transaction Confirmation Behavior Is Unspecified
Category: Functional Reliability / Usability — Requirement to Define

The mock-up includes a Continue button, but it does not clarify what happens after submission, how processing failures are handled, or how the user is informed of the transaction outcome.

Recommendation: Define the payment flow for successful transactions, declined payments, processing errors, and timeouts. Provide clear feedback so users can understand whether their payment was completed or requires further action.

9. Payment Test Environment and Test Data Are Not Specified
Category: Testability — Requirement to Define

The mock-up does not specify how the payment functionality should be tested safely. Using real card details during testing could expose sensitive information or result in unintended transactions.

Recommendation: Define the payment provider's test environment and use its designated test card numbers for automated tests. Ensure that test transactions cannot trigger real charges.

B. Sample Test Cases

Test Case 1 — Successful Payment Form Submission

Objective: Verify that the billing form accepts valid information and allows the user to proceed through the payment flow.

Preconditions: The billing form is accessible, and the payment provider's test environment is configured.

Test Data:

Field	Value
Card Type	VISA
Payment Amount	30.00
Card Number	Valid test card number provided by the payment provider
Expiration Month	12
Expiration Year	2028
First Name	John
MI	A
Last Name	Doe
Billing Street Address	123 Main Street
Address Line 2	Apt 4B
City	New York
State or Province	NY
Postal Code	10001

Test Steps:

1. Open the billing form in the test environment.
2. Enter the test data into the corresponding fields.
3. Click Continue.
4. Observe the resulting payment flow and confirmation message.

Expected Result: The form accepts the valid test data and proceeds through the payment flow. The user receives clear confirmation of the successful outcome, and the transaction is recorded as successful in the test environment. No validation errors are displayed.

Test Case 2 — Required Fields and Invalid Input Validation

Objective: Verify that the form identifies missing or invalid information and prevents progression when the input does not meet the defined validation rules.

Preconditions: The billing form is accessible, and the required-field and input-validation rules have been defined.

Test Data:

Field	Value
Card Type	VISA
Payment Amount	30.00
Card Number	123
Expiration Month	12
Expiration Year	2020
First Name	(empty)
MI	A
Last Name	Doe
Billing Street Address	(empty)
Address Line 2	(empty)
City	New York
State or Province	NY
Postal Code	ABC

Test Steps:

1. Open the billing form.
2. Enter the test data into the corresponding fields.
3. Click Continue.
4. Observe the validation messages and verify whether the payment flow proceeds.

Expected Result: The form prevents progression and displays clear validation messages for the invalid or missing information. The invalid card number and expired expiration date are rejected, required empty fields are identified, and the postal code is validated according to the rules for the selected country. No payment is processed.

Test Case 3 — Prevent Duplicate Payment Submissions

Objective: Verify that repeated clicks on Continue do not result in duplicate payment transactions.

Preconditions: The billing form is accessible, and the payment provider's test environment is configured.

Test Data: Use the valid test data from Test Case 1, including a test card number provided by the payment provider.

Test Steps:

1. Open the billing form in the test environment.
2. Complete all fields using the valid test data.
3. Click Continue repeatedly in quick succession.
4. Inspect the payment requests and transaction records in the test environment.
5. Observe the feedback displayed to the user while the payment is being processed.

Expected Result: Only one payment transaction is processed for the submission. Repeated clicks or requests do not create duplicate charges. The interface provides appropriate feedback

while processing and clearly communicates the final transaction outcome.

C. Product Solution for the Most Severe Issue

Selected Problem: Duplicate Payment Submissions

Rationale for Selecting the Problem

Duplicate payment submissions were selected as the primary issue for this proposal because they can cause direct financial harm to customers and create additional costs and operational work for the business. Although the mock-up does not confirm that this vulnerability exists, the risk is relevant to the payment flow and can be addressed through specific, testable safeguards.

Payment data security is equally important and could have more severe consequences if a vulnerability were confirmed. However, the mock-up does not provide enough information to identify a specific security flaw or determine how payment information is handled. Therefore, duplicate payments offer a more concrete starting point for a targeted product solution based on the information available.

Proposed Product Solution

Implement a multi-layered protection mechanism to prevent duplicate payment transactions while keeping users informed of the payment status.

1. Frontend Protection

When the user clicks Continue, temporarily disable the button and display a loading indicator or message, such as *Processing payment....* This reduces accidental repeated submissions and provides immediate feedback that the request is being processed.

2. Backend Idempotency

2. Backend Idempotency

Implement an idempotency mechanism to prevent repeated requests for the same payment operation from creating duplicate charges. Each logical payment attempt should be associated with a unique idempotency key. If the same attempt is retried, the key must be reused so that the backend or payment provider can return the original result instead of processing another charge.

This protection is essential because frontend controls alone cannot prevent duplicate requests caused by network retries, timeouts, or other unexpected behavior.

3. Payment Status and User Feedback

Provide clear feedback for successful payments, declined transactions, and processing errors. If a request times out or the outcome is uncertain, the system should verify the transaction status before allowing another payment attempt. This prevents users from unknowingly submitting a second payment when the first request may already have succeeded.

Expected Results

- Reduce the risk of duplicate charges caused by repeated submissions or request retries.
- Improve the user experience by providing clear feedback during payment processing.

- Reduce avoidable refund requests, customer support workload, and operational costs.
- Avoid unnecessary payment processing caused by duplicate requests, improving operational efficiency.

Validation Criteria

The solution should be validated through automated tests that simulate rapid repeated clicks, repeated requests using the same idempotency key, and network timeouts where the payment may have succeeded despite the missing response.

The expected outcome is that each intended payment operation results in no more than one charge, and that the user receives an accurate status for the transaction.